

组原第四次实验报告

实验名称：ALU 实验及改进

学号：2312966 姓名：林晖鹏 班次：张金老师

一、实验目的

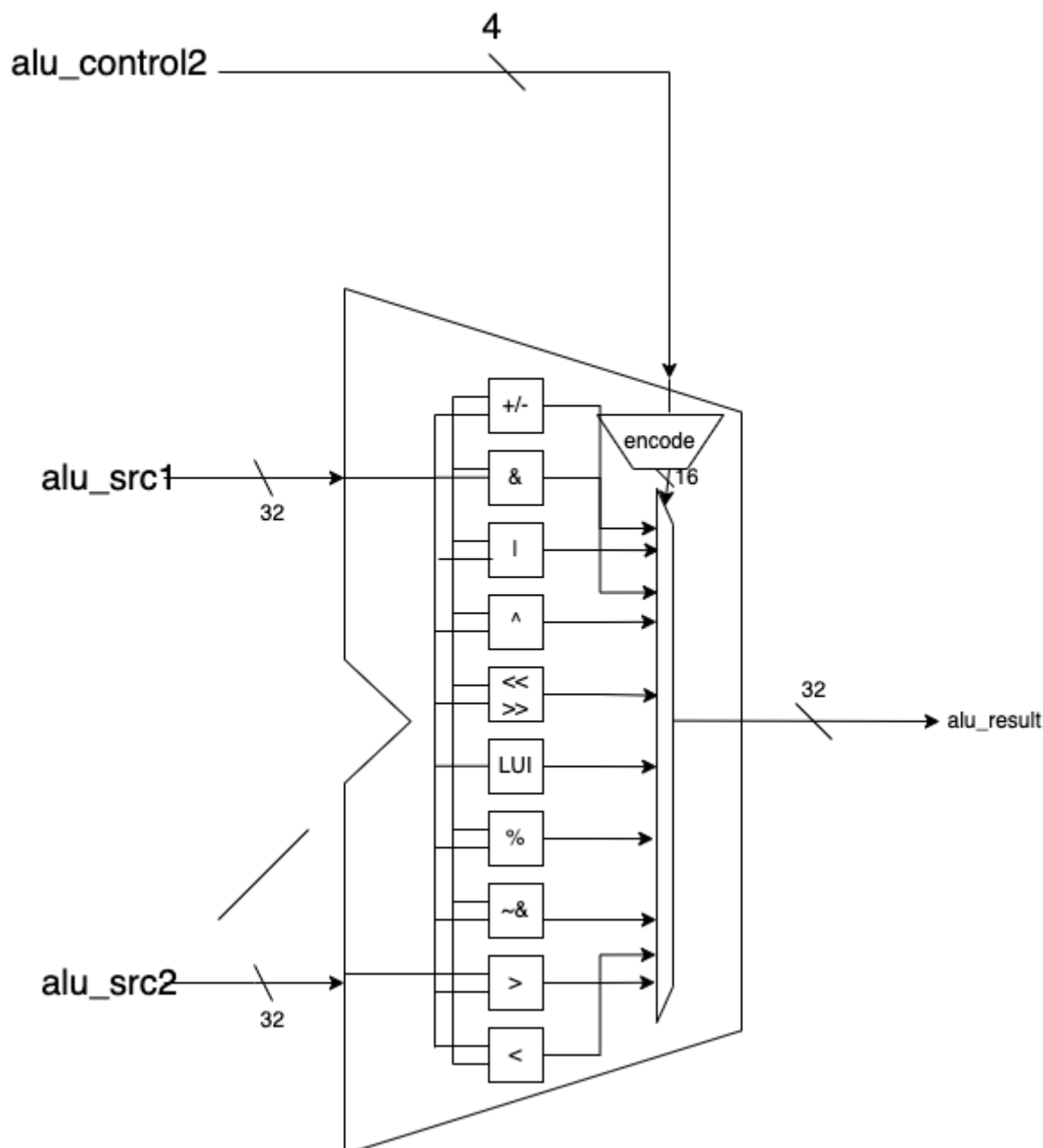
- 熟悉并掌握 ALU 的原理、功能和设计。
- 进一步加强运用 verilog 语言进行电路设计的能力。

二、实验内容

- 将原有的操作码进行位压缩，调整操作码控制信号位宽为 4 位。
- 操作码调整成 4 位之后，在原有 12 种运算的基础之上，补充 4 种不同类型的运算：**无符号数大于置位、有符号数大于置位、与非、取模运算。**
- 上实验箱验证实际功能。

三、实验原理图

四位二进制的选择操作数输入之后，先进入一个编码器，编码成16位二进制的独热编码，在进行原版的ALU操作运算。



四、实验步骤

1. 操作码压缩

这里要求将原本的独热编码压缩成四位操作码，我们修改成 4 位二进制数，通过数字的大小来控制操作功能。

1.1 修改顶层模块和 `alu` 模块输入

我们修改顶层模块的输入。

代码块

```
1  reg    [3:0] alu_control; // ALU控制信号
```

修改 `alu` 模块的输入：

代码块

```
1  module alu(  
2      input [3:0] alu_control2,  
3      //input [11:0] alu_control, // ALU控制信号  
4      input [31:0] alu_src1,      // ALU操作数1,为补码  
5      input [31:0] alu_src2,      // ALU操作数2,为补码  
6      output [31:0] alu_result    // ALU结果  
7  );
```

1.2 编码器设计

我们不想改动原本代码的框架，我们下面设计一个编码器，输入为四位二进制，输出为原框架中使用的独热编码：

代码块

```
1  reg [15:0] alu_control;  
2  always @(*) begin  
3      case (alu_control2)  
4          4'b0000: alu_control = 16'b0000_0000_0000_0001;  
5          4'b0001: alu_control = 16'b0000_0000_0000_0010;  
6          4'b0010: alu_control = 16'b0000_0000_0000_0100;  
7          4'b0011: alu_control = 16'b0000_0000_0000_1000;  
8          4'b0100: alu_control = 16'b0000_0000_0001_0000;  
9          4'b0101: alu_control = 16'b0000_0000_0010_0000;  
10         4'b0110: alu_control = 16'b0000_0000_0100_0000;  
11         4'b0111: alu_control = 16'b0000_0000_1000_0000;  
12         4'b1000: alu_control = 16'b0000_0001_0000_0000;  
13         4'b1001: alu_control = 16'b0000_0010_0000_0000;  
14         4'b1010: alu_control = 16'b0000_0100_0000_0000;  
15         4'b1011: alu_control = 16'b0000_1000_0000_0000;  
16         4'b1100: alu_control = 16'b0001_0000_0000_0000;  
17         4'b1101: alu_control = 16'b0010_0000_0000_0000;  
18         4'b1110: alu_control = 16'b0100_0000_0000_0000;  
19         4'b1111: alu_control = 16'b1000_0000_0000_0000;  
20         default: alu_control = 16'b0000_1000_0000_0000;  
21     endcase  
22 end
```

模块输入 `alu_control2` 之后，用 `case` 语句判定操作数，输出操作位的独热编码。

2. 添加功能变量

按照其他功能添加对应变量。

代码块

```
1  wire alu_and_not;    //按位与非
2  wire alu_rem;        // a%b
3  wire alu_big;        //有符号比较，大于置位，复用加法器做减法
4  wire alu_bigu;       //无符号比较，大于置位，复用加法器做减法
5
6      .....
7
8  assign alu_and_not  = alu_control[15];
9  assign alu_rem      = alu_control[14];
10 assign alu_big      = alu_control[13];
11 assign alu_bigu     = alu_control[12];
12
13      .....
14
15 wire [31:0] and_not_result;
16 wire [31:0] big_result;
17 wire [31:0] bigu_result;
18 wire [31:0] rem_result;
```

3. 与非和取模运算功能实现

直接公式运算得到对应功能结果。

代码块

```
1  assign and_not_result = ~(alu_src1 & alu_src2);    // 与结果为两数按位与非
2  assign rem_result    = alu_src1 % alu_src2;        // 取模
```

4. 大于置位功能实现

我们仿照小于置位功能，相当于小于置位的取反，但是这里我们还要排除掉**相等**的情况。

我们用一个 wire 变量 `equ` 来判断两个操作数是否相等，方便在后面排除相等的情况。

然后就是按照小于置位的功能取反来实现大于置位。

下面是 big 情况表：

adder_src1[31]	adder_src2[31]	adder_result[31]	情况

0	1	X (0 或 1)	"正-负", 显然大于
0	0	1	相减为负, 说明小于
0	0	0	相减为正, 说明大于等于
1	1	1	相减为负, 说明小于
1	1	0	相减为正, 说明大于等于
1	0	X (0 或 1)	"负-正", 显然小于成立

代码块

```

1 //判断两个是否相等
2 wire equ = adder_result == 32'd0 ? 1:0;
3 assign big_result[31:1] = 31'd0;
4 //正-负 | (~负-正 & ~相等 & ~)
5 assign big_result[0] = (~alu_src1[31] & alu_src2[31]) | (~
  (alu_src1[31]^alu_src2[31]) & ~equ & ~adder_result[31]);
6
7 wire bigger = adder_cout & (~equ);
8 assign bigu_result = {31'd0, adder_cout & (~equ)};

```

5. 添加对应结果输出

代码块

```

1 // 选择相应结果输出
2 assign alu_result = (alu_add|alu_sub) ? add_sub_result[31:0] :
3     alu_slt ? slt_result :
4     alu_sltu ? sltu_result :
5     alu_and ? and_result :
6     alu_nor ? nor_result :
7     alu_or ? or_result :
8     alu_xor ? xor_result :
9     alu_sll ? sll_result :
10    alu_srl ? srl_result :
11    alu_sra ? sra_result :
12    alu_lui ? lui_result :
13    alu_and_not ? and_not_result :
14    alu_rem ? rem_result :
15    alu_big ? big_result :
16    alu_bigu ? bigu_result :
17    32'd0;

```

五、实验结果分析

1. 仿真实验

下面我们来设计一个测试文件来测试对应功能，并输出部分信息：

代码块

```
1  module tb;
2      reg [31:0] alu_src1, alu_src2;
3      wire [31:0] alu_result;
4      reg [3:0] ctr;
5
6
7      alu a(
8          .alu_control2(ctr),
9          .alu_src1(alu_src1),
10         .alu_src2(alu_src2),
11         .alu_result(alu_result)
12     );
13
14
15     initial begin
16         $display("Starting ALU tests with randomized inputs...");
17
18         ctr = 4'b1111;
19
20         repeat(10) begin
21             alu_src1 = $random;
22             alu_src2 = $random;
23
24             $display("Test %d: alu_src1 = %b, alu_src2 = %b", $time, alu_src1,
25                 alu_src2);
26
27             #10;
28             $display("Test %d Result: alu_result = %b", $time, alu_result);
29
30             #5;
31         end
32
33         $finish;
34     end
35
36     initial begin
```

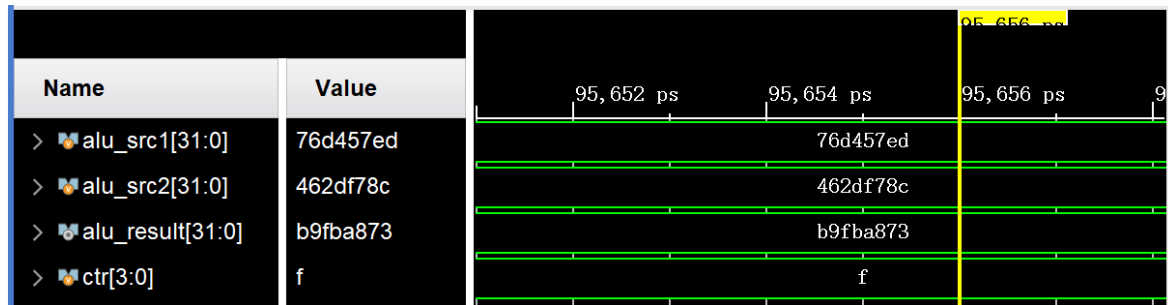
```

36         $monitor("Time = %0t: ALU Control = %b, alu_src1 = %b, alu_src2 = %b,
Result = %b", $time, ctr, alu_src1, alu_src2, alu_result);
37     end
38 endmodule

```

每次测试的时候，修改 ctr 的数值就能测试不同功能，下面我们以**与非功能**为例，此时 $ctr == 1111$ 。

仿真结果如下，经过验证，结果无误。



2. 上箱实验

约束文件已经写好了，我们只需要直接上箱。这里值得注意的是，在约束文件中，选择操作数的按钮位置十分偏僻，为左数第二个和右数最后一个按钮。

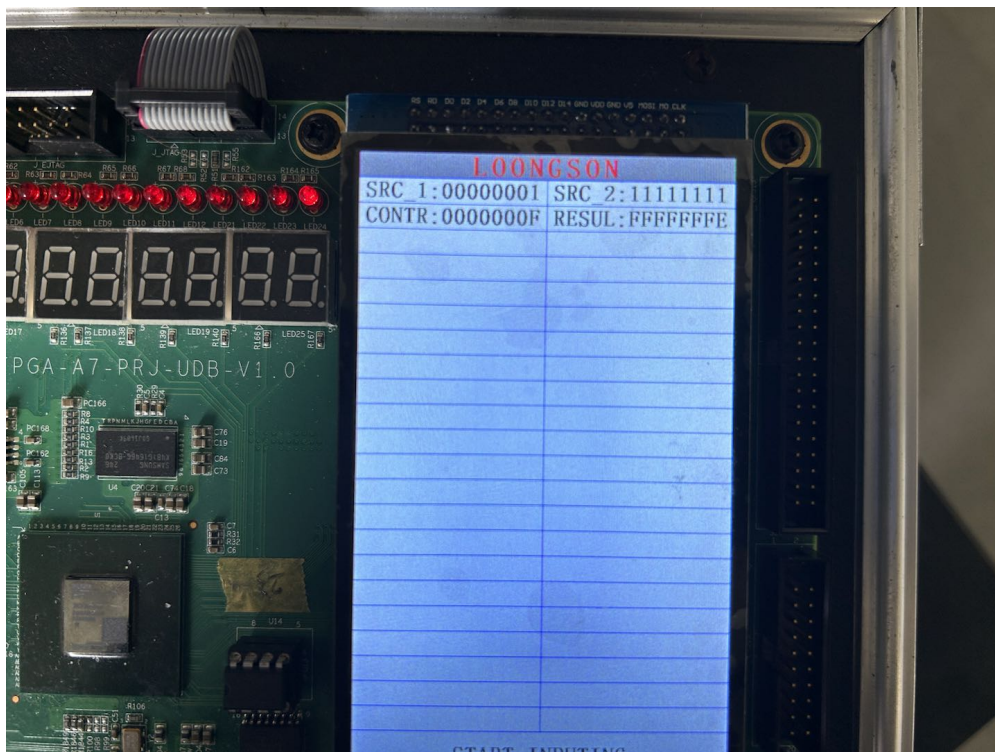
2.1 用加法验证基本功能使用

我们尝试用加法功能在验证基础使用。



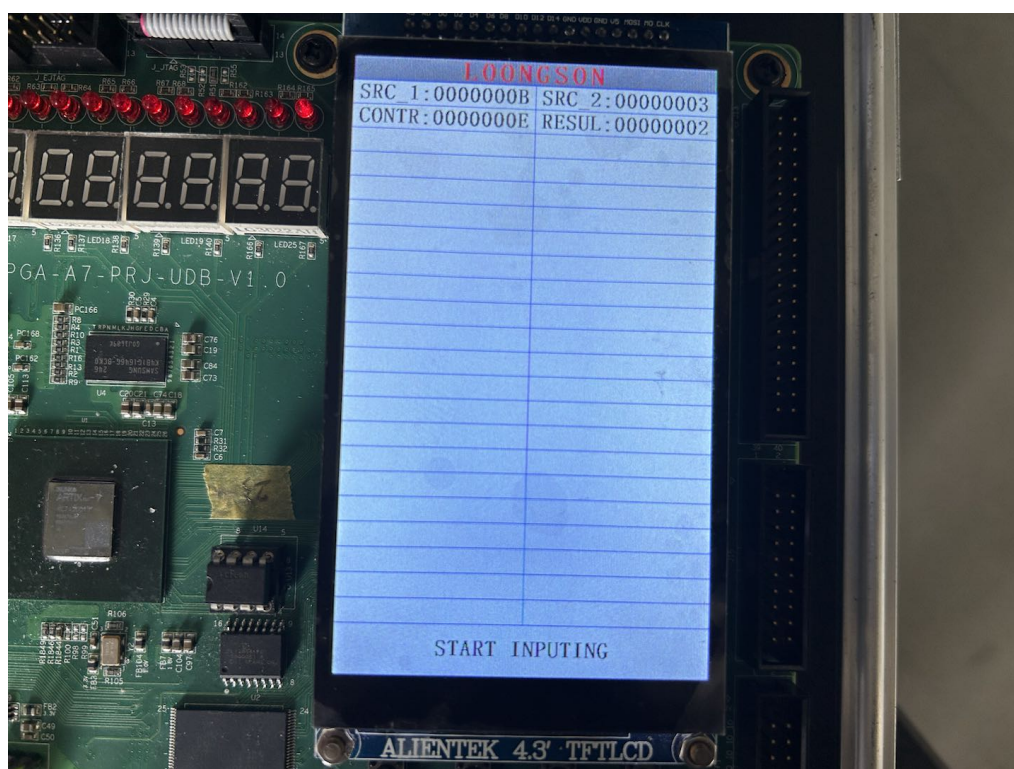
2.2 验证与非功能

下面输入{F}选择与非功能，输入操作数{00000001}，{11111111}，输出{FFFFFFFE}。验证成功。



2.3 验证取模功能

下面输入{E}选择取模功能，输入操作数{0000000B}，{00000003}，输出{00000002}。验证成功。
这里发现，当我模数为 0 的时候，他不会更新结果，防止了错误情况的出现。



2.4 验证大于置位功能

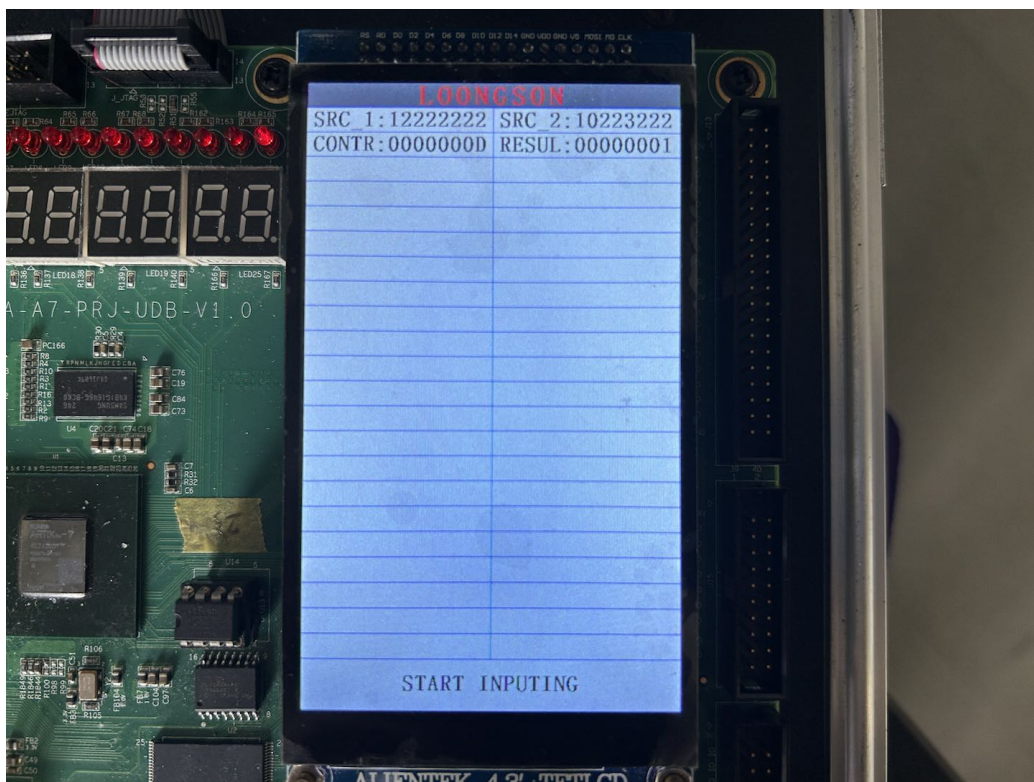
无符号数大于置位：

下面输入{C}，输入操作数{00000055}，{00000011}，输出{00000001}。验证成功。



有符号数大于置位：

下面输入{D}, 输入操作数{12222222}, {10223222}, 输出{00000001}。验证成功。



六、总结心得

- 本次实验我们学习了 ALU 设计框架及其代码，并且实现了四种功能的添加，对 ALU 器件的理解更加透彻了。

- 在模拟仿真和上箱的时候发现，如果在 ALU 中独热编码只有 15 位，但是在仿真的时候，仍然能实现与非功能的（与非功能的独热编码为 1000_0000_0000_0000）仿真，但是在上箱的时候并不能实现与非功能。导致我在 debug 的时候根本找不到问题所在，比较头疼。
-